

Computational Systems Biology Final Assignment

April 24, 2026

David Lewis

lewis3d7@mail.uc.edu

Figure 1 (the replication of Figure 2 in Dovzhenok et al.) was the easiest to create, notably, as long as the initial conditions were above a certain threshold, they eventually settled into this oscillatory behavior. I used the DifferentialEquations package in Julia for Figure 1 and it was very straightforward. Plotting is done using the Plots package.

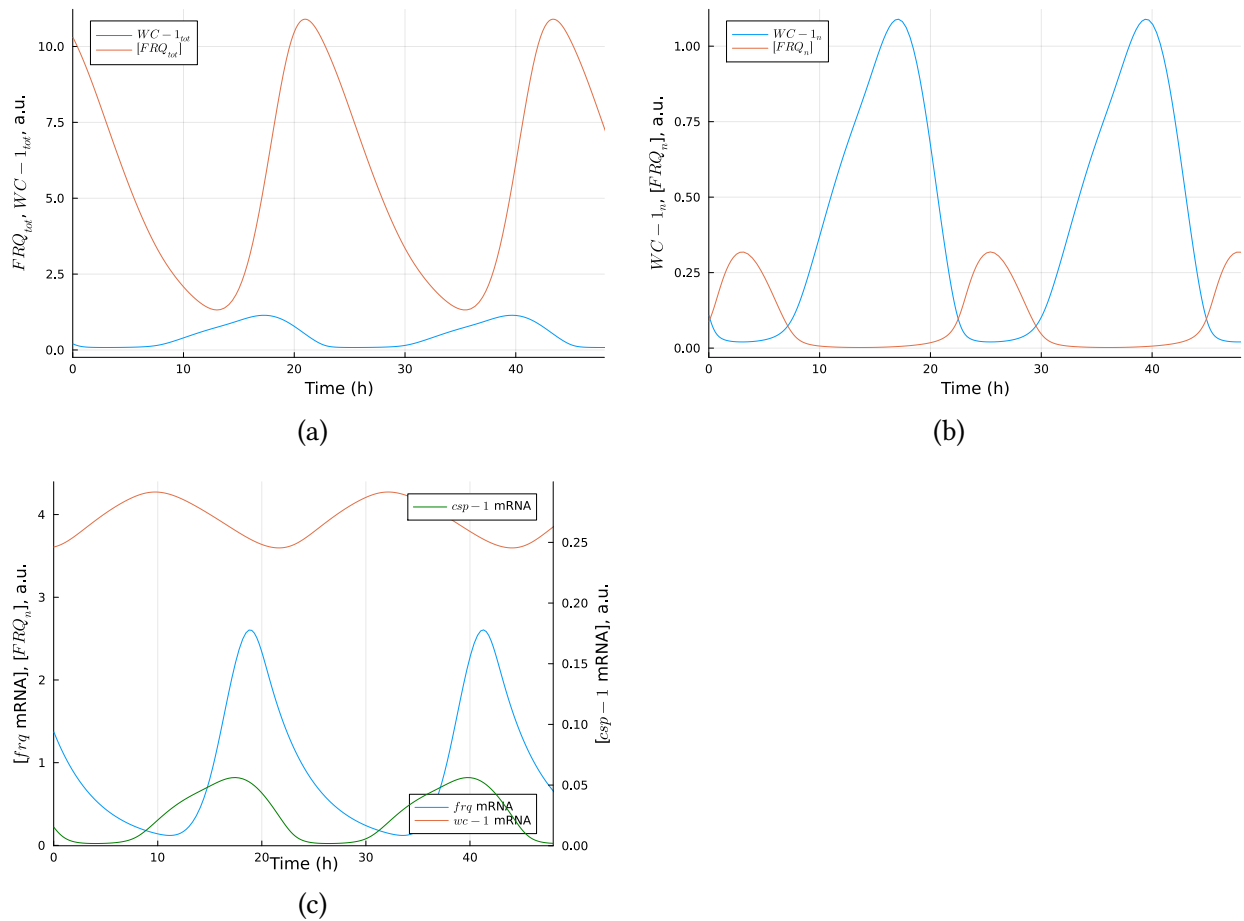


Figure 2 was not so easy to create. I used the BifurcationKit package for this and followed the general procedure for these plots. First, I found the Hopf points, then used BifurcationKit to simulate the model using parameter values along a given range. Since there is no Hopf point for Figure 2c, I instead sampled a simulation (as in Figure 1) as the “initial guess” for BifurcationKit. Finding the Hopf points was not very intuitive as whatever algorithm BifurcationKit uses seems to be very sensitive to initial conditions. So if k_{03} for example was not in a particular range, then the Hopf point would not be found, even though it exists. I found out that if I boosted the initial conditions, by setting the initial concentrations to large values, the algorithm became less sensitive to these parameter changes, something which was necessary to reproduce Figure 3.

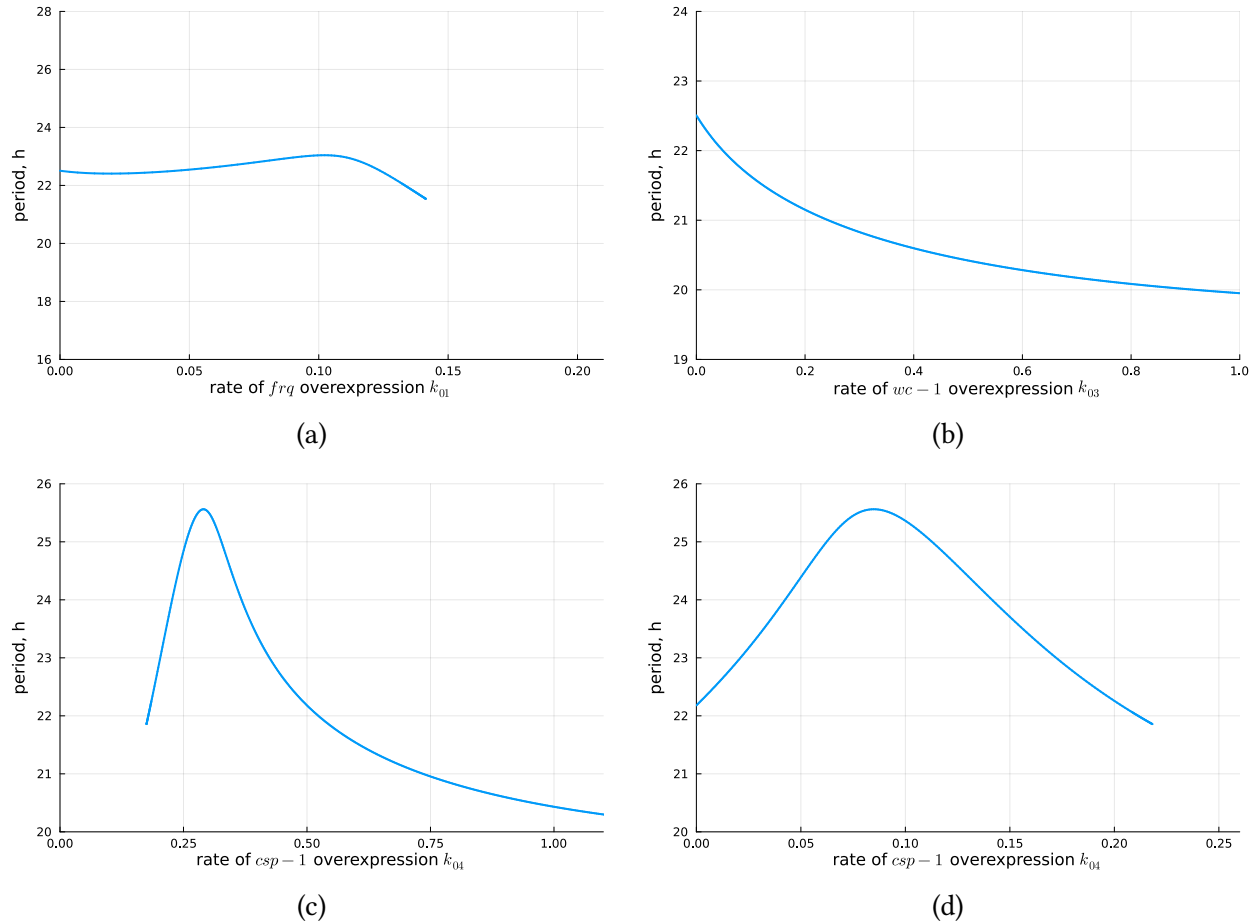
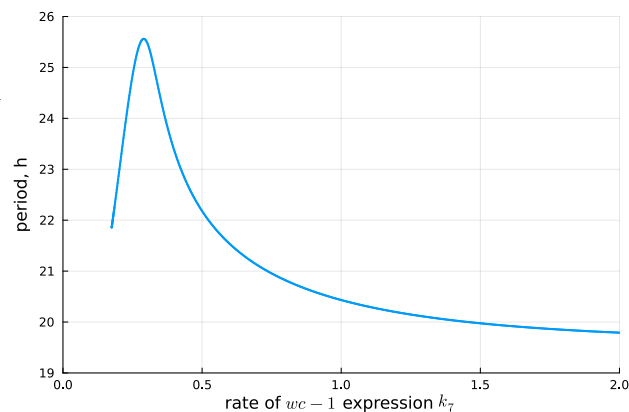


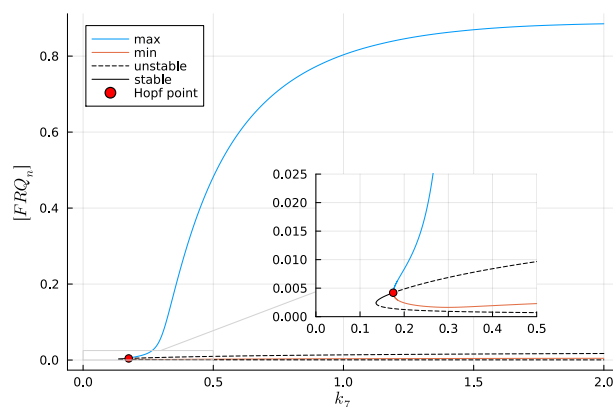
Figure 2: Reproduction of Figure 3

Figure 3 uses essentially the same process to reproduce the figure, except the black lines, as in Figure 3c, needed to be extracted from the function that found the Hopf points. Luckily, these values were fairly easy to find in the generated object.

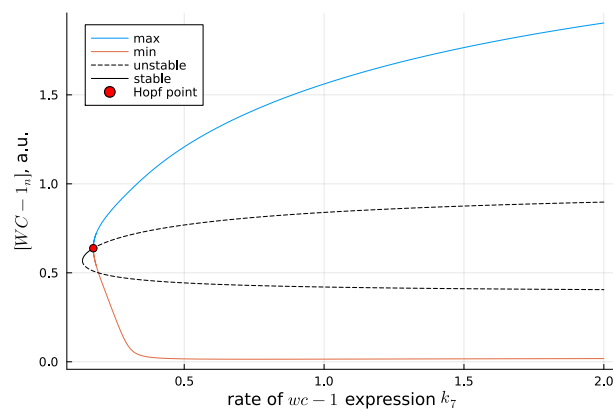
To note, I needed to set all the initial concentrations to 10 to get the full range of the black lines in Figure 3b and Figure 3c for some reason, otherwise the Hopf points would fail to be calculated.



(a)



(b)



(c)

Figure 3: Reproduction of Figure

4

Code

Nicer code link: <https://illustratedman-code.github.io/compsys-final/>

Julia

```

1 ### A Pluto.jl notebook ###
2 # v0.20.13
3
4 using Markdown
5 using InteractiveUtils
6
7 # ╰══ cecd1f2a-0c8c-405b-9323-9e84707bfb54
8 begin
9     import DifferentialEquations as DE
10    using Plots
11    using LaTeXStrings
12    import BifurcationKit as BK
13    import BifurcationKit: @optic
14    using Accessors #for dealing with updating params tuple in a not horrible way
15    md"""
16    Imports
17    """
18 end
19
20 # ╰══ eaa483e0-4e41-4dca-86e8-9f5eb92a73d1
21 begin
22    k = (1.8, 1.8, 0.05, 0.23, 0.27, 0.27, 0.5, 1, 40, 0.1, 0.05, 0.02, 50, 1, 5,
23         0.12, 1.4, 50, 1.4)
24    # these are 0 indexed because the authors hate me :),
25    # so add 1 to all indicies compared to the paper. e.g. K = K[1] K_1 = K[2] etc
26    K = (1.25, 3, 1, 10, 3)
27
28    k0 = (one = 0.0, three = 0.0, four = 0.0)
29
30    md"""
31    Parameters
32    """
33
34 end
35
36 # ╰══ cf1818b8-63d1-4551-b521-734609b304c6
37 begin
38    const DUI = (
39        frq_mRNA = 1, FRQ_c = 2, FRQ_n = 3,
40        wc1_mRNA = 4, WC1_c = 5, WC1_n = 6,
41        FRQ_n_WC1_n = 7, csp1_mRNA = 8, CSP1 = 9
42    )
43
44    function model(du, u, p, t)
45        k, k0, K = p.k, p.k0, p.K
46
47        # Map u to local variables (zero allocation, AD-safe)
48        f_mRNA = u[1]; FRQ_c = u[2]; FRQ_n = u[3]
49        w_mRNA = u[4]; WC1_c = u[5]; WC1_n = u[6]
50        Complex = u[7]; c_mRNA = u[8]; CSP1 = u[9]
51
52        # Math logic (Types will promote naturally here)
53        du[DUI.frq_mRNA] = k[1] * ((WC1_n^6)/(K[1] + WC1_n^6)) - k[4] * f_mRNA +
54            k0.one

```

Julia

```

55     du[DUI.FRQ_c] = k[2] * f_mRNA - (k[3] + k[5]) * FRQ_c
56
57     du[DUI.FRQ_n] = k[3] * FRQ_c + k[14] * Complex - FRQ_n * (k[6] + k[13] *
WC1_n)
58
59     du[DUI.wc1_mRNA] = k[7] * (K[2]/(K[2] + CSP1)) - k[10] * w_mRNA + k0.three
60
61     du[DUI.WC1_c] = k[8] * ((FRQ_c^2)/(K[3] + FRQ_c^2)) * (w_mRNA/(K[4] +
w_mRNA)) - (k[9] + k[11]) * WC1_c
62
63     du[DUI.WC1_n] = k[9] * WC1_c - WC1_n * (k[12] + k[13] * FRQ_n) + k[14] *
Complex
64
65     du[DUI.FRQ_n_WC1_n] = k[13] * FRQ_n * WC1_n - (k[14] + k[15]) * Complex
66
67     du[DUI.csp1_mRNA] = k[16] * WC1_n * (K[5]/(K[5] + CSP1)) - k[17] * c_mRNA +
k0.four
68
69     du[DUI.CSP1] = k[18] * c_mRNA - k[19] * CSP1
70 end
71
72 end
73
74 # 7f8c2040-0362-4db5-904a-739881c55c67
75 begin
76     u0 = fill(1.0, 9) # need to give the initial condition at least a 1 to oscillate
77     tspan = (0.0, 3000)
78
79     params = (k=k, K=K, k0=k0)
80
81     problem = DE.ODEProblem(model, u0, tspan, params)
82
83     original_sol = DE.solve(problem)
84
85     sol = original_sol(original_sol.t[original_sol.t .>= 500])
86
87 end
88
89 # 51fbb9cb-fec1-43d6-9123-96adcabf3d16
90 solution = Dict{"frq mRNA" => sol[1, :],
91                "FRQ_c" => sol[2, :],
92                "FRQ_n" => sol[3, :],
93                "wc-1 mRNA" => sol[4, :],
94                "WC-1_c" => sol[5, :],
95                "WC-1_n" => sol[6, :],
96                "FRQ_n:WC-1_n" => sol[7, :],
97                "csp-1 mRNA" => sol[8, :],
98                "CSP-1" => sol[9, :],
99                "WC-1_tot" => sol[5, :] + sol[6, :] + sol[7, :],
100               "FRQ_tot" => sol[2, :] + sol[3, :] + sol[7, :],
101               "time" => sol.t .- 500
102            )
103
104
105 # f2981768-ea50-4881-83dd-a85f1b0461f6
106 md"""

```

Julia

```

107 Figure 2A
108 """
109
110 # 81d3cc77-6695-42bf-8ef4-9e7fe18373d8
111 begin
112     figure2A = plot(solution["time"], solution["WC-1_tot"], label=L"WC-1_{tot}",
113                     xlims=(0, 48))
113     plot!(solution["time"], solution["FRQ_tot"], label=L"FRQ_{tot}", xlims=(0,
114     48))
114     # plot(solution["time"], solution["WC-1_tot"], label=L"WC-1_{tot}")
115     # plot!(solution["time"], solution["FRQ_tot"], label=L"FRQ_{tot}")
116     xlabel!("Time (h)")
117     ylabel!(L"$FRQ_{tot}$, $WC-1_{tot}$, a.u.")
118     savefig("Figure-2A.svg")
119     figure2A
120 end
121
122 # b584625a-641d-4d8b-9d96-05b9323771d4
123 md"""
124 Figure 2B
125 """
126
127 # 5a45f599-996d-4b56-a52d-9004b1c9fa85
128 begin
129     figure2B = plot(solution["time"], solution["WC-1_n"], label=L"WC-1_n", xlims=(0,
130     48))
130     plot!(solution["time"], solution["FRQ_n"], label=L"FRQ_n", xlims=(0, 48))
131     xlabel!("Time (h)")
132     ylabel!(L"$WC-1_n$, [FRQ_n], a.u.")
133     savefig("Figure-2B.svg")
134     figure2B
135 end
136
137 # 43ae3d5f-6604-4002-b5a3-84e0215421e4
138 md"""Figure 2C"""
139
140 # 23434a30-8901-4115-9d30-87f7f3799f16
141 begin
142     figure2C = plot(solution["time"], solution["frq mRNA"], label=L"$frq$ mRNA",
143                     xlims=(0, 48))
143     plot!(solution["time"], solution["wc-1 mRNA"], label=L"$wc-1$ mRNA", xlims=(0,
144     48))
144     figure2C_twin = twinx(figure2C)
145     plot!(figure2C_twin, solution["time"], solution["csp-1 mRNA"], label=L"$csp-1$
146     mRNA", xlims=(0, 48), ylims=(0, 0.3), legend=true, color="green")
147     xlabel!(figure2C_twin, "Time (h)")
148     ylabel!(L"$frq$ mRNA", [FRQ_n], a.u.")
149     ylabel!(figure2C_twin, L"$csp-1$ mRNA", a.u.")
150     savefig("Figure-2C.svg")
151     figure2C
152 end
153
154
155 # 33dcd342-73ef-4a4c-bcff-be9ca322394a

```

Julia

```

156 function bifurcation_model(u, p)
157     param_type = typeof(p.k0.one) ## these screw up type inference for some reason
158     param_type2 = typeof(p.k0.four) ## so we promote the type
159     param_type3 = typeof(p.k0.three)
160     param_type4 = typeof(p.k[7])
161
162     # Promote between the state 'u' and the parameter type
163     T = promote_type(eltype(u), param_type)
164     T = promote_type(T, param_type2)
165     T = promote_type(T, param_type3)
166     T = promote_type(T, param_type4)
167
168     # Create du with the promoted type
169     du = similar(u, T)
170
171     model(du, u, p, 0.0)
172     return du
173 end
174
175
176 # 1116de3d-2887-4147-bcf1-748a1b922a2f
177 begin
178     u_end = sol.u[end-1] # this doesn't give hopf points in figure4A for some reason
179     u_end = fill(1.0, 9) # have to just set everything to one instead
180     md"""setting initial conditions"""
181 end
182
183 # 4c1cda2a-b40b-4061-b462-871345c56a74
184 function calculate_period(optic, params=params; u=u_end, step_direction=1,
    return_br=false)
185     # define the bifurcation based on our parameter (i.e. k01)
186     prob = BK.BifurcationProblem(bifurcation_model, u, params, optic)
187
188     # Establish the sweep for the k0 param
189     k0_range = BK.ContinuationPar(
190         p_min = 0.0,
191         p_max = 2.0,
192         ds = 0.01 * step_direction,
193         dsmax = 0.01, # ensures that we get lots of points for plotting
194         detect_bifurcation = 3,
195         detect_fold= true,
196         max_steps = 1800
197     )
198
199     # calculate hopf points
200     br = BK.continuation(prob, BK.PALC(), k0_range)
201     plot(br)
202     savefig("br.svg")
203     print(br)
204     if length(br.specialpoint) < 2
205         # ideally we only get two special points, the end point and the hopf point
206         # if not, then return this object for debugging
207         print("There are no hopf points")
208         return br
209     end
210

```

Julia

```

211     if return_br
212         return br
213     end
214
215     # trapezoidal estimation of period orbits
216     # this isn't exactly equivalent to xppaut, but close enough
217     # and it's much faster to compute
218     period_prob = BK.PeriodicOrbitTrapProblem(M=200)
219
220     # compute the period vs param based on the hopf point
221     period_branch = BK.continuation(
222         br, 1, # Ensure '1' is the index of the hopf point
223         k0_range,
224         period_prob;
225         nev=1000,
226     )
227
228
229     return period_branch
230
231 end
232
233
234 # faf2d03b-fc6c-41a1-8c17-5e57a7b7409b
235 md"""
236 Figure 3A
237 """
238
239 # b3de8be2-61c0-42cb-84d4-2746f8b7ef5c
240 period_branch = calculate_period((@optic _.k0.one))
241
242 # 18c7b721-165d-4ffa-bc3b-f96d279d1a4f
243 begin
244
245     figure3A = plot(period_branch, vars = (:param, :period),
246         xlabel = L"rate of $frq$ overexpression $k_{01}$",
247         ylabel = "period, h",
248         xlims = (0, 0.21),
249         ylims = (16, 28)
250     )
251     savefig("Figure-3A.svg")
252     figure3A
253 end
254
255 # 78e71262-628d-4d41-a61d-50c0526b4908
256 function period_no_hopf(optic, parameters=params)
257
258     one_period_problem = DE.ODEProblem(model, original_sol.u[end], (0, 25),
259         params)
260     one_period_sol = DE.solve(one_period_problem)
261     prob_bif = BK.ODEBifProblem(bifurcation_model, u_end, parameters, optic;)
262     k03_range = BK.ContinuationPar(
263         p_min = 0.0,
264         p_max = 1.0,
265         ds = 0.01,
266         dsmax = 0.01

```


Julia

```

266 )
267 period_prob = BK.PeriodicOrbitTrapProblem(M=200)
268 print(parameters)
269 probtrap, ci = BK.generate_ci_problem(period_prob,
270                                     prob_bif, one_period_sol, (0, 22))
271 opts_po_cont = BK.ContinuationPar(k03_range, max_steps = 1800, tol_stability =
1e-5)
272 brpo_fold = BK.continuation(probtrap, ci, BK.PALC(), opts_po_cont;
273                             verbosity = 3, plot = true)
274 return brpo_fold
275 end
276
277
278 # 791547fc-f77f-4713-a3cd-e7e158150a53
279 brpo_fold = period_no_hopf(@optic _.k0.three)
280
281 # 4c02ee18-0135-43f6-b8b2-ab3751b8bc28
282 begin
283 figure3B = plot(brpo_fold, vars = (:param, :period),
284                xlabel = L"rate of $wc-1$ overexpression $k_{03}$",
285                ylabel = "period, h",
286                xlims = (0, 1),
287                ylims = (19, 24)
288                )
289 savefig("Figure-3B.svg")
290 figure3B
291 end
292
293 # 398ab915-d292-4fe8-8de6-4250b676e7df
294 begin
295 k7_0_params = @set params.k[7] = 0.0
296 k7_0_params = @set k7_0_params.k0.three = 0.5
297 end
298
299 # c3475b10-a4cd-4ccd-8d89-c117f30e47e7
300 period_branch_figure_3c = calculate_period((@optic _.k0.three), k7_0_params;
step_direction=-1)
301
302
303 # 159d2889-43e6-43c5-82c7-972f6ab03b9b
304 begin
305 figure3C = plot(period_branch_figure_3c, vars = (:param, :period),
306                xlabel = L"rate of $csp-1$ overexpression $k_{04}$",
307                ylabel = "period, h",
308                xlims = (0, 1.1),
309                ylims = (20, 26)
310                )
311 savefig("Figure-3C.svg")
312 figure3C
313 end
314
315 # df93de58-a1cb-4ee4-a3e2-2217c631661e
316 k16_0_params = @set params.k[16] = 0.0
317
318 # 8e545e6e-c917-4666-a72f-133b1a3f5cf7
319 period_branch_figure_3D = calculate_period((@optic _.k0.four), k16_0_params)

```

Julia

```

320
321 #  e91148ff-6acc-4be9-9c29-bee646bded34
322 begin
323     figure3D = plot(period_branch_figure_3D, vars = (:param, :period),
324                     xlabel = L"rate of $csp-1$ overexpression $k_{04}$",
325                     ylabel = "period, h",
326                     xlims = (0, 0.26),
327                     ylims = (20, 26)
328                     )
329     savefig("Figure-3D.svg")
330     figure3D
331 end
332
333 #  b4cfe200-9ac6-4218-b633-593506894a3f
334 begin
335     k7_params = @set params.k[7] = 0.5
336     k7_params = @set k7_params.k[16] = 0
337 end
338
339 #  f25b1eb8-0abb-40ed-9960-71e6795b7119
340 period_branch_figure_4A = calculate_period((@optic _.k[7]), k7_params;
341     step_direction=-1)
342
343 #  dc41454d-2964-494a-925c-dac4c0747520
344 begin
345     figure4A = plot(period_branch_figure_4A, vars = (:param, :period),
346                     xlabel = L"rate of $wc-1$ expression $k_{7}$",
347                     ylabel = "period, h",
348                     xlims = (0, 2.0),
349                     ylims = (19, 26)
350                     )
351     savefig("Figure-4A.svg")
352     figure4A
353 end
354
355 #  440d406e-cf56-47c9-8e99-9b911c9270b2
356 function plot_one_condition_vs_param(condition, period_branch)
357     max_u = []
358     min_u = []
359     for i_sol in 1:length(period_branch)
360         traj = BK.get_periodic_orbit(period_branch, i_sol)
361         push!(max_u, maximum(traj[condition, :]))
362         push!(min_u, minimum(traj[condition, :]))
363     end
364     p = plot(period_branch.param, max_u; label="max")
365     plot!(period_branch.param, min_u; label="min")
366     return p
367 end
368
369 end
370
371
372 #  1d002879-e74b-4551-9b5c-52b1e8c4f161
373 function plot_one_condition_steady_vs_param(condition, period_branch)
374     p = []

```

Julia

```

375     value = []
376     for i_sol in 1:length(period_branch.sol)
377         traj = period_branch.sol[i_sol]
378         push!(p, traj.p)
379         push!(value, traj.x[condition])
380
381     end
382     ps = [[p[1]]]
383     values = [[value[1]]]
384     stable = [period_branch.stable[1] == false ? "unstable" : "stable"]
385     for i in range(2, length(period_branch.stable))
386         if period_branch.stable[i] == true && period_branch.stable[i-1] == true
387             push!(ps[end], p[i])
388             push!(values[end], value[i])
389         elseif period_branch.stable[i] == false && period_branch.stable[i-1] == false
390             push!(ps[end], p[i])
391             push!(values[end], value[i])
392         else
393             push!(stable, period_branch.stable[i] == false ? "unstable" : "stable")
394             push!(ps, [p[i]])
395             push!(values, [value[i]])
396         end
397     end
398
399     seen_labels = Set{String}()
400     for (p_i, val, stable_i) in zip(ps, values, stable)
401         stable_label = stable_i ∈ seen_labels ? "" : stable_i
402
403         if stable_i == "unstable"
404             plot!(p_i, val; label=stable_label, linestyle = :dash, color="black")
405             push!(seen_labels, stable_i)
406         else
407             plot!(p_i, val; label=stable_label, color="black")
408             push!(seen_labels, stable_i)
409         end
410     end
411     scatter!([p[period_branch.specialpoint[1].idx]],
412             [value[period_branch.specialpoint[1].idx]], color="red", label="Hopf point" )
413     #stability = [x == false ? "unstable" : "stable" for x in period_branch.stable]
414     #p = plot!(p, value; group=stability, linestyle = :dash)
415     return stable
416 end
417
418 # 2dbdf652-4f2c-4bb6-b55d-d89ef0efb2c4
419 begin
420     k7_params_large = @set params.k[7] = 2.0
421     k7_params_large = @set k7_params_large.k[16] = 0
422 end
423
424 # cb7efa53-b940-4588-ab12-36189081d98d
425 period_br = calculate_period((@optic _.k[7]), k7_params_large; u=fill(10.0, 9),
426                             step_direction=-1, return_br=true)
427
428 # b6b7dc51-4aac-4419-97d3-f77d45447ebb

```

Julia

```
429 md"""Figure 4B"""
430
431 #  c257c7c9-e801-40f4-9318-b6b42f1dd77e
432 begin
433 plot_4B = plot_one_condition_vs_param(DUI.FRQ_n, period_branch_figure_4A)
434 plot_one_condition_steady_vs_param(DUI.FRQ_n, period_br)
435 xlabel!(L"k_7")
436 ylabel!(L"[FRQ_n]")
437 lens!(plot_4B, [0.0, 0.5], [0.0, 0.025], inset = (1, bbox(0.45, 0.45, 0.4, 0.4)))
438 savefig("Figure-4B.svg")
439 plot_4B
440 end
441
442
443 #  b176e2d7-0f59-4978-91fe-1550f73d437e
444 md"""
445 Figure 4C
446 """
447
448 #  d05c838a-eca8-4c20-a7b1-e9b83283a721
449 begin
450 figure4C = plot_one_condition_vs_param(DUI.WC1_n, period_branch_figure_4A)
451 plot_one_condition_steady_vs_param(DUI.WC1_n, period_br)
452 xlabel!(L"rate of $wc-1$ expression $k_7$")
453 ylabel!(L"$[WC-1_n]$, a.u.")
454 savefig("Figure-4C.svg")
455 figure4C
456 end
457
```